

AZURE_PROVIDER

Azure OpenAI Provider - Implementation Guide

This document provides a comprehensive guide to the Azure OpenAI Provider implementation in HelixCode.

Overview

The Azure OpenAI Provider enables access to OpenAI models through Microsoft Azure OpenAI Service. This provider supports all OpenAI models (GPT-4, GPT-3.5, o1, embeddings, etc.) deployed on Azure infrastructure with enterprise features including:

- **Microsoft Entra ID Authentication:** Secure authentication using Azure Active Directory
- **Deployment-based Routing:** Map model names to Azure deployment names
- **API Versioning:** Support for latest Azure OpenAI API versions
- **Content Filtering:** Built-in Azure content filtering with detailed error reporting
- **Streaming Support:** Server-Sent Events (SSE) for real-time responses
- **Token Caching:** Automatic Entra ID token caching with refresh
- **Managed Identity:** Support for system and user-assigned managed identities

Implementation Files

Core Implementation

- **azure_provider.go:** Main provider implementation (963 lines)
 - Provider interface implementation
 - Azure SDK integration
 - Deployment mapping
 - Authentication (API key and Entra ID)
 - Request/response handling
 - Streaming support
 - Content filtering
 - Error handling
- **azure_provider_test.go:** Comprehensive test suite (643 lines)
 - 20 test cases covering all functionality

- Mock HTTP server tests
- Authentication tests (API key and Entra ID)
- Deployment mapping tests
- Content filtering tests
- Streaming tests
- Error handling tests
- Health check tests

Configuration Files

- **config/azure_example.yaml**: Example configuration demonstrating all options
- **provider.go**: Updated with ProviderTypeAzure constant and factory support

Architecture

Provider Structure

```
type AzureProvider struct {
    config          ProviderConfigEntry
    apiKey          string
    endpoint        string           // e.g., https://
                   myresource.openai.azure.com
    apiVersion      string           // e.g., 2025-04-01-preview
    deploymentMap   map[string]string // model name ->
                   deployment name
    httpClient      *http.Client
    models          []ModelInfo
    entraTokenProvider *EntraTokenProvider // for Entra ID auth
    lastHealth      *ProviderHealth
}
```

Entra Token Provider

```
type EntraTokenProvider struct {
    credential azcore.TokenCredential
    tokenCache *string
    tokenExpiry time.Time
    mutex      sync.RWMutex
}
```

Features: - Thread-safe token caching - Automatic token refresh (5 minutes before expiry) - Double-checked locking pattern - Support for any Azure credential type

Supported Models

GPT-4 Family

- gpt-4-turbo - Latest GPT-4 model with 128K context
- gpt-4 - Standard GPT-4 with 8K context
- gpt-4-32k - Extended context window (32K)
- gpt-4-vision-preview - Multimodal capabilities
- gpt-4o - Latest multimodal model
- gpt-4o-mini - Fast and efficient variant

GPT-3.5 Family

- gpt-35-turbo - Fast and cost-effective
- gpt-35-turbo-16k - Extended context

o1 Reasoning Models

- o1-preview - Advanced reasoning (128K context, 32K output)
- o1-mini - Faster reasoning (128K context, 16K output)

Embedding Models

- text-embedding-3-large - Latest large embeddings
- text-embedding-ada-002 - Classic embedding model

All models support: - Text generation - Code generation and analysis - Planning and debugging - Refactoring and testing - Tools/function calling (except o1 models)
- Vision capabilities (selected models)

Configuration

Basic API Key Authentication

```
llm:
  providers:
    azure:
      type: "azure"
      enabled: true
      api_key: "${AZURE_OPENAI_API_KEY}"
      endpoint: "https://myresource.openai.azure.com"
      api_version: "2025-04-01-preview"
      deployment_map:
        gpt-4-turbo: "my-gpt4-deployment"
```

```
    gpt-35-turbo: "my-gpt35-deployment"
models:
  - "gpt-4-turbo"
  - "gpt-35-turbo"
```

Entra ID Authentication

```
llm:
  providers:
    azure:
      type: "azure"
      enabled: true
      endpoint: "https://myresource.openai.azure.com"
      api_version: "2025-04-01-preview"
      use_entra_id: true # Enable Entra ID
      deployment_map:
        gpt-4-turbo: "production-gpt4"
```

Managed Identity

```
llm:
  providers:
    azure:
      type: "azure"
      enabled: true
      endpoint: "https://myresource.openai.azure.com"
      use_entra_id: true
      managed_identity: true
      # Optional: for user-assigned identity
      managed_identity_client_id:
        "00000000-0000-0000-0000-000000000000"
```

Environment Variables

```
# API Key Authentication
export AZURE_OPENAI_API_KEY="your-api-key"
export AZURE_OPENAI_ENDPOINT="https://
  myresource.openai.azure.com"
export AZURE_API_VERSION="2025-04-01-preview"

# Deployment map (JSON string or file path)
export AZURE_DEPLOYMENTS_MAP='{"gpt-4-turbo": "my-deployment"}'
# OR
export AZURE_DEPLOYMENTS_MAP="/path/to/deployments.json"
```

```
# Entra ID Authentication
export AZURE_TENANT_ID="your-tenant-id"
export AZURE_CLIENT_ID="your-client-id"
export AZURE_CLIENT_SECRET="your-client-secret"
```

Deployment Mapping

Azure OpenAI uses deployment names instead of model names. The provider supports multiple ways to configure this mapping:

1. Inline Map (YAML)

```
deployment_map:
  gpt-4-turbo: "my-gpt4-deployment"
  gpt-35-turbo: "my-gpt35-deployment"
```

2. JSON String

```
deployment_map: '{"gpt-4-turbo": "my-deployment"}'
```

3. External JSON File

```
deployment_map: "/etc/helixcode/azure-deployments.json"
```

File contents:

```
{
  "gpt-4-turbo": "production-gpt4-deployment",
  "gpt-35-turbo": "production-gpt35-deployment",
  "gpt-4o": "gpt4o-deployment"
}
```

4. Environment Variable

```
export AZURE_DEPLOYMENTS_MAP='{"gpt-4-turbo": "my-deployment"}'
```

Fallback Behavior

If no mapping is found for a model, the provider uses the model name as the deployment name. This works when your Azure deployments are named identically to the models.

Usage Examples

Basic Generation

```
import "dev.helix.code/internal/llm"

// Initialize provider
config := llm.ProviderConfigEntry{
    Type:    llm.ProviderTypeAzure,
    APIKey:  os.Getenv("AZURE_OPENAI_API_KEY"),
    Parameters: map[string]interface{}{
        "endpoint":    "https://myresource.openai.azure.com",
        "api_version": "2025-04-01-preview",
        "deployment_map": map[string]string{
            "gpt-4-turbo": "my-gpt4-deployment",
        },
    },
}

provider, err := llm.NewAzureProvider(config)
if err != nil {
    log.Fatal(err)
}
defer provider.Close()

// Generate response
request := &llm.LLMRequest{
    ID:    uuid.New(),
    Model: "gpt-4-turbo",
    Messages: []llm.Message{
        {Role: "user", Content: "Explain Azure OpenAI Service."},
    },
    MaxTokens: 500,
    Temperature: 0.7,
}

response, err := provider.Generate(context.Background(), request)
if err != nil {
    log.Fatal(err)
}

fmt.Println(response.Content)
fmt.Printf("Tokens used: %d\n", response.Usage.TotalTokens)
```



```

        "type": "string",
        "description": "City name",
    },
},
    "required": []string{"location"},
},
},
},
},
}

response, err := provider.Generate(context.Background(), request)
if err != nil {
    log.Fatal(err)
}

// Check for tool calls
for _, toolCall := range response.ToolCalls {
    fmt.Printf("Tool: %s\n", toolCall.Function.Name)
    fmt.Printf("Args: %v\n", toolCall.Function.Arguments)
}

```

Entra ID Authentication

```

import (
    "github.com/Azure/azure-sdk-for-go/sdk/azidentity"
    "dev.helix.code/internal/llm"
)

// Create credential
credential, err := azidentity.NewDefaultAzureCredential(nil)
if err != nil {
    log.Fatal(err)
}

// Create Entra token provider
entraProvider := llm.NewEntraTokenProvider(credential)

// Create Azure provider
provider := &llm.AzureProvider{
    endpoint: "https://myresource.openai.azure.com",
    apiVersion: "2025-04-01-preview",
    entraTokenProvider: entraProvider,
    deploymentMap: loadDeploymentMap(),
}

```



```

    httpClient:      &http.Client{Timeout: 120 * time.Second},
    models:          llm.getAzureModels(),
}

// Use normally
response, err := provider.Generate(context.Background(), request)

```

Content Filtering

Azure OpenAI includes built-in content filtering that analyzes both prompts and completions for:

- Hate speech
- Self-harm
- Sexual content
- Violence

Severity levels: safe, low, medium, high

When content is filtered, the provider returns a descriptive error:

```

_, err := provider.Generate(ctx, request)
if err != nil {
    if strings.Contains(err.Error(), "content filtered") {
        // Handle content filtering
        fmt.Println("Content was blocked by Azure filtering")
    }
}

```

Error message format:

content filtered by Azure: prompt contains prohibited content (hate=medium)

Error Handling

The provider handles Azure-specific errors:

Error Codes

- `content_filter` → Content filtering error
- `DeploymentNotFound` → `ErrModelNotFound`
- `InvalidRequestError` → `ErrInvalidRequest`
- `RateLimitError, 429` → `ErrRateLimited`
- `QuotaExceeded` → Quota exceeded error
- `InvalidApiKey, Unauthorized` → Authentication error

HTTP Status Codes

- 401 → Unauthorized (check API key/token)
- 403 → Forbidden (check permissions)
- 404 → Model/deployment not found
- 429 → Rate limited
- 400 → Invalid request

Example

```
response, err := provider.Generate(ctx, request)
if err != nil {
    switch {
    case errors.Is(err, llm.ErrRateLimited):
        // Implement exponential backoff
        time.Sleep(time.Second * 5)
        // Retry...

    case errors.Is(err, llm.ErrModelNotFound):
        // Check deployment mapping
        fmt.Println("Deployment not found - check your
        deployment_map")

    case strings.Contains(err.Error(), "content filtered"):
        // Content was filtered
        fmt.Println("Content blocked by Azure")

    default:
        log.Printf("Error: %v", err)
    }
}
```

Health Checks

The provider implements health checking:

```
health, err := provider.GetHealth(context.Background())
if err != nil {
    log.Printf("Provider unhealthy: %v", err)
}

fmt.Printf("Status: %s\n", health.Status)
fmt.Printf("Latency: %v\n", health.Latency)
fmt.Printf("Models: %d\n", health.ModelCount)
```

Health check performs a minimal request to verify: - Connectivity to Azure endpoint - Authentication validity - Deployment availability

Testing

The implementation includes 20 comprehensive test cases:

```
# Run all Azure provider tests
```

```
go test -v ./internal/llm -run TestAzureProvider
```

```
# Run specific test
```

```
go test -v ./internal/llm -run TestAzureProvider_Generate_APIKey
```

```
# Run with coverage
```

```
go test -cover ./internal/llm -run TestAzureProvider
```

Test Coverage

1. Provider initialization (API key)
2. Provider initialization (Entra ID)
3. Missing endpoint validation
4. Explicit deployment mapping
5. Fallback deployment mapping
6. JSON deployment mapping
7. Basic generation with API key
8. Content filtering error
9. Rate limit error
10. Deployment not found error
11. Streaming generation
12. Entra token caching
13. Provider type and name
14. Models and capabilities
15. IsAvailable check
16. Health check success
17. Health check failure
18. Provider close
19. Tool support
20. Default max tokens

Dependencies

```
require (  
    github.com/Azure/azure-sdk-for-go/sdk/azcore v1.16.0  
    github.com/Azure/azure-sdk-for-go/sdk/azidentity v1.8.0
```

github.com/google/uuid v1.6.0
)

API Versioning

The provider defaults to 2025-04-01-preview (latest stable). Supported versions:

- 2025-04-01-preview (recommended)
- 2024-10-01-preview
- 2024-08-01-preview
- 2024-06-01

Specify in configuration:

`api_version: "2025-04-01-preview"`

Performance Characteristics

- **Latency:** Similar to OpenAI (200ms - 2s first token)
- **Throughput:** Based on Azure deployment tier
- **Rate Limits:** Configurable per deployment (TPM/RPM)
- **Regional Availability:** Deploy in regions close to users
- **SLA:** 99.9% uptime for Standard tier

Security Considerations

1. **Managed Identity:** Use Azure managed identities for production
2. **Private Endpoints:** Enable private endpoints for VNet isolation
3. **Content Filtering:** Configure content filters based on use case
4. **RBAC:** Use Azure RBAC for fine-grained access control
5. **Audit Logging:** Enable diagnostic logging for compliance
6. **Key Rotation:** Implement regular API key rotation
7. **Network Security:** Use Azure Firewall rules to restrict access

Migration from OpenAI

Key Changes

1. **Endpoint:** Change from `api.openai.com` to Azure resource endpoint
2. **Authentication:** Use `api-key` header instead of `Authorization: Bearer`
3. **API Version:** Add `api-version` query parameter
4. **Deployments:** Map model names to deployment names
5. **URL Format:** `/openai/deployments/{deployment}/chat/completions`

Migration Steps

1. Deploy models in Azure OpenAI portal
2. Create deployment mappings
3. Update configuration to use Azure provider
4. Test with health check
5. Monitor content filtering

Troubleshooting

Common Issues

Issue: “deployment not found” - **Solution:** Check deployment_map configuration - Verify deployment exists in Azure portal - Ensure deployment name is correct

Issue: “unauthorized: check API key” - **Solution:** Verify API key is correct - Check key is set in environment variable - For Entra ID, check credentials are valid

Issue: “content filtered by Azure” - **Solution:** Review and modify prompt - Adjust content filtering settings in Azure portal - Check which category triggered the filter

Issue: “rate limited” - **Solution:** Implement exponential backoff - Increase deployment quota in Azure - Use multiple deployments for load distribution

Issue: Entra ID token expired - **Solution:** Token automatically refreshes - Check Azure credentials are valid - Verify token scopes are correct

Best Practices

1. **Use Entra ID:** Prefer Entra ID over API keys for production
2. **Deployment Naming:** Use descriptive deployment names
3. **Error Handling:** Implement retry logic for rate limits
4. **Monitoring:** Track token usage and latency
5. **Content Filtering:** Configure appropriate filters for your use case
6. **Regional Deployment:** Deploy close to your users
7. **Health Checks:** Monitor provider health regularly
8. **Caching:** Provider automatically caches Entra ID tokens
9. **Timeouts:** Configure appropriate HTTP timeouts (default 120s)
10. **Versioning:** Pin API version for stability

References

- [Azure OpenAI Documentation](#)

- [Azure OpenAI API Reference](#)
- [Azure SDK for Go](#)
- [Content Filtering](#)
- [Technical Design Document](#)

Support

For issues or questions: 1. Check this documentation 2. Review test cases for examples 3. Check Azure OpenAI service health 4. Review Azure portal deployment status 5. Enable debug logging for detailed errors